



python

Apprendre
à coder
avec



Pr. Houda BENHAR
Année universitaire 2023-2024

PLAN

- Introduction
- Classe
- Encapsulation
- Héritage
- Polymorphisme



INTRODUCTION

- La Programmation Orientée Objet (POO) est un paradigme de programmation qui repose sur le concept d'**objets**, des entités qui regroupent à la fois des données (également appelées **attributs**) et des fonctionnalités (appelées **méthodes**) qui opèrent sur ces données.
- Les concepts **clés** de la programmation orientée objet (POO) comprennent :
 - ❖ **Encapsulation**
 - ❖ **Héritage**
 - ❖ **Polymorphisme**
- la POO favorise la **modularité**, la **réutilisation** du code, et la **structuration logique** des programmes en les organisant autour d'objets interagissant entre eux



LE CONCEPT DE CLASSE

- Une **classe** est un **type** ou **modèle** de définition pour des entités (ou objets) ayant les mêmes **caractéristiques** (ou **attributs**) et les mêmes comportements (ou **méthodes**)
- Un **objet** est alors une **variable** de la classe: on dit qu'il est une **instance** de cette classe, c'est-à-dire un élément concret, un exemplaire de cette classe.
- Une classe est composée de membres :
 - des attributs qui donnent l'état des instances
 - des méthodes qui indiquent les opérations qui pourront être effectuées sur les instances
 - d'un constructeur pour créer les instances



LE CONCEPT DE CLASSE

1) Syntaxe de déclaration d'une classe:

```
class NomDeClasse:  
    # contenu
```

Exemple:

```
class Animal:  
    pass
```

Instanciación:

```
animal1 = Animal()  
animal2 = Animal()
```



LE CONCEPT DE CLASSE

2) Les constructeurs: un constructeur est une méthode spéciale appelée **`__init__()`** qui est utilisée pour initialiser les attributs d'une classe lors de la création d'instances de cette classe.

```
def __init__(self, [arg1, [arg2] ...]):  
    self.attribut1 = arg1
```

Exemple:

```
class Animal:  
    def __init__(self):  
        print("un animal vient d'être créé")
```

```
animal1 = Animal()  
animal2 = Animal()
```



LE CONCEPT DE CLASSE

3) **Les attributs**: un attribut est une variable associée à une classe ou à un objet. Il représente une propriété ou une caractéristique de la classe ou de l'objet.

- Les **attributs d'instance** sont des variables définies à l'aide de **self**. Elles sont relatives à l'instance, et ne peuvent être accédées sans instantiation.
- Les **attributs de classe** sont des variables définies directement dans le corps de la classe. Elles peuvent être accédées par la classe, sans passer par l'instanciation.

Exemple 1:

```
class Animal:
    def __init__(self):
        print("un animal vient d'être créé")
        self.age = 0

animal1 = Animal()
# pour accéder à un attribut d'un objet on utilise un point .
print(animal1.age)
animal1.age = 25
```

Exemple 2:

```
class Animal:
    nb_animaux_crees = 0 # attribut de classe
    def __init__(self, nom, espece):
        self.nom = nom # attribut d'instance
        self.espece = espece
        self.age = 0

chat = Animal("Minou", "Chat")
print(chat.age)
print(chat.nom)
```



LES MÉTHODES

- Une méthode est une fonction définie à l'intérieur d'une classe. Il existe différents types de méthodes que vous pouvez définir dans une classe:

1) Méthode d'instance: elle prend toujours **self** comme premier paramètre, qui fait référence à l'instance de l'objet sur lequel la méthode est appelée.

Exemple:

```
class Animal:
```

```
    def __init__(self, nom, espece, age = 0):  
        self.nom = nom  
        self.espece = espece  
        self.age = age
```

```
    def sePresenter(self):  
        print(f"Bonjour, je m'appelle {self.nom} et je suis un {self.espece} de {self.age} ans.")
```

```
chat = Animal("Minou", "Chat", 1)  
chat.sePresenter()
```



LES MÉTHODES

2) Méthode de classe: c'est une méthode qui est associée à la classe elle-même plutôt qu'à une instance particulière de cette classe

- Elle est définie en utilisant le décorateur **@classmethod**.
- Les méthodes de classe peuvent accéder aux attributs de classe mais ne peuvent pas accéder aux attributs d'instance car elles ne sont pas liées à une instance spécifique.

Exemple:

```
class Animal:
    nb_animaux_crees = 0
    def __init__(self):
        self.age = 0
        Animal.nb_animaux_crees += 1

    @classmethod
    def getNbAnimauxCrees(cls):
        return cls.nb_animaux_crees

print(Animal.getNbAnimauxCrees())
```



LES MÉTHODES

3) Méthode statique: une méthode statique est une méthode qui n'est pas liée à une instance spécifique ou à la classe elle-même.

- Elle est définie en utilisant le décorateur **@staticmethod**
- Elle peut être appelée à la fois par la classe elle-même et par les instances de cette classe, mais elle n'a pas accès aux attributs de l'instance.

Exemple:

```
class Animal:
    def __init__(self):
        self.age = 0

    @staticmethod
    def est_vieux(age_limite, animal_age):
        return animal_age >= age_limite

age_limite = 10
animal1 = Animal()
if Animal.est_vieux(age_limite, animal1.age):
    print("Cet animal est considéré comme vieux.")
else:
    print("Cet animal n'est pas considéré comme vieux.")
```



ENCAPSULATION

- La capacité de regrouper les données et les méthodes qui opèrent sur ces données dans une seule entité est appelée encapsulation. Cela permet de cacher les détails d'implémentation et de protéger les données sensibles.
- L'encapsulation permet de définir une interface claire pour interagir avec un objet, en limitant l'accès direct aux données internes de cet objet.
- Cela favorise la modularité, la réutilisabilité et la facilité de maintenance du code, car les détails de l'implémentation sont masqués à l'utilisateur de la classe.
- Pour restreindre l'accès direct aux membres d'une classe on utilise les **Accesseurs** et les **Mutateurs**:
 - Les accesseurs (**getters**) sont des méthodes qui permettent de retourner la valeur d'un attribut. Les mutateurs (**setters**) permettent de modifier la valeur d'un attribut.



ENCAPSULATION

- **Encapsulation des attributs:** en POO, on distingue différents types d'attributs: les attributs **publics**, les attributs **protégés** et les attributs **privés**.
- Un attribut privé n'est accessible qu'à l'intérieur de la définition de la classe: je ne pourrais y accéder (pour lecture ou écriture) que dans la définition des différentes méthodes. A contrario, les attributs publics sont accessibles partout et toujours.
- Pour définir un attribut privé, on va nommer cet attribut en commençant par __

```
class Animal:  
    def __init__(self, age = 0):  
        self.__age = age
```

```
animal = Animal()  
print(animal.age)
```

Traceback (most recent call last):

File "[C:\Users\h-oud\PycharmProjects\functions\mon_module.py](#)", line 8, in <module>

print(animal.age)

AttributeError: 'Animal' object has no attribute 'age'



ENCAPSULATION

- **Encapsulation des attributs:** access à un attribut privé en lecture et en écriture à l'aide d'un **accesseur** et un **mutateur** respectivement

```
class Animal:  
    def __init__(self, age = 0):  
        self.__age = age
```

```
    def getAge(self):  
        return self.__age
```

```
    def setAge(self, value):  
        self.__age = value
```

```
animal1 = Animal()  
animal1.setAge(0.5)  
print(animal1.getAge())
```



ENCAPSULATION

- **Encapsulation des attributs:** access à un attribut privé à l'aide d'un accesseur

```
class Animal:
    def __init__(self):
        self.__age = 0

    @property
    def getAge(self):
        return self.__age

    def vieillir(self):
        self.__age = self.__age + 1

animal = Animal()
animal.vieillir()
print(animal.getAge)
```



Un accesseur est une méthode qui renvoie la valeur d'un attribut d'instance. Il se définit avec le décorateur @property



ENCAPSULATION

- **Encapsulation des attributs:** modification d'un attribut privé à l'aide d'un mutateur

```
class Animal:
    def __init__(self):
        self.__age = 0

    @property
    def getAge(self):
        return self.__age

    @getAge.setter
    def setAge(self, value):
        self.__age = value

animal = Animal()
animal.setAge = 0.5
print(animal.getAge)
```



Un mutateur est une méthode qui change la valeur d'un attribut d'instance. Il se définit avec le décorateur `@property.setter`



HÉRITAGE

- L'héritage est l'un des concepts fondamentaux de la programmation orientée objet (POO). Il permet de créer de nouvelles classes basées sur des classes existantes.
- L'héritage permet de réutiliser le code en définissant des fonctionnalités communes dans la classe parente.
- Quand nous créons notre classe, nous allons lui passer en paramètre le nom de la classe dont nous voulons hériter.

```
class MaClasseFille(MaClasseMere):
```

```
...
```

- Puis, dans notre constructeur, nous allons initialiser le constructeur de notre classe mère avant d'initialiser notre propre classe.

```
class MaClasseFille(MaClasseMere):
```

```
    def __init__(self):
```

```
        MaClasseMere.__init__(self) # super().__init__(self)
```

```
...
```



HÉRITAGE

- Si une classe hérite d'une autre classe, elle hérite toutes ses attributs et méthodes.
- **Exemple 1:**

```
class Animal:  
    def __init__(self, nom, espece, age=0):  
        self.nom = nom  
        self.espece = espece  
        self.age = age
```

```
class Chat(Animal):  
    pass
```

```
mon_chat = Chat('Lily', 'Chat')
```



HÉRITAGE

- **Exemple 2:**

```
class Animal:
    def __init__(self, nom, espece, age=0):
        self.nom = nom
        self.espece = espece
        self.age = age

    def setAge(self, value):
        self.age = value

class Chat(Animal):
    pass

mon_chat = Chat('Lily', 'Chat')
print(mon_chat.age)
mon_chat.setAge(0.5)
print(mon_chat.age)
```



HÉRITAGE

- **Exemple 3:**

```
class Animal:
    def __init__(self, nom, espece, age=0):
        self.nom = nom
        self.espece = espece
        self.age = age

class Chat(Animal):
    def __init__(self, nom, age):
        super().__init__(nom, espece="Chat", age=age)

chat = Chat('Lily', 0.5)
```



POLYMORPHISME

- Dans le contexte de la programmation orientée objet, le polymorphisme est un concept qui fait référence à la capacité d'une variable, d'une fonction ou d'un objet à prendre plusieurs formes, c'est-à-dire à sa capacité de posséder plusieurs définitions différentes.
- En **Python**, le **polymorphisme** sert à définir une méthodes dans la classe enfant avec le **même nom qu'une méthodes de la classe mère**.
- Le principe de **l'héritage polymorphe** est d'offrir la possibilité à une classe enfant d'hériter une méthode de la classe mère et d'y ajouter des modifications selon le besoin.



POLYMORPHISME

- **Exemple:**

```
class Animal:
    def __init__(self, nom, espece, age=0):
        self.nom = nom
        self.espece = espece
        self.age = age
    def parle(self):
        print("Je peux parler!")

class Chat(Animal):
    def __init__(self, nom, age):
        super().__init__(nom, espece="chat", age=age)

    def parle(self):
        print("Meow!")

chat = Chat('Lily', 0.5)
chat.parle()
```



MÉTHODES MAGIQUES

- Les méthodes spéciales en Python, également connues sous le nom de méthodes du double souligné (ou méthodes magiques), sont des méthodes prédéfinies qui peuvent être utilisées pour personnaliser le comportement des objets dans des contextes spécifiques.
- **Exemple: méthode `__str__`**

```
class Animal:
    def __init__(self, nom, espece, age = 0):
        self.nom = nom
        self.espece = espece
        self.age = age
    #def se_presenter(self):
    #    #print(f"Bonjour, je m'appelle {self.nom} et je suis un {self.espece} de {self.age} ans.")

    def __str__(self):
        return "Bonjour, je m'appelle {} et je suis un {} de {} ans".format(self.nom, self.espece, self.age)
```

```
animal1 = Animal('Lily', 0.5)
print(animal1)
```



MÉTHODES MAGIQUES

Exemple: méthodes `__lt__` et `__eq__`

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __lt__(self, other):
        return self.x < other.x and self.y < other.y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

p1 = Point(1, 2)
p2 = Point(1, 2)
print(p1 == p2) # True
print(p1 < p2) # False
```



MÉTHODES MAGIQUES

`__ne__(self, other)`: Cette méthode est appelée lorsqu'un objet est comparé à un autre à l'aide de l'opérateur `!=`.

`__lt__(self, other)`: Cette méthode est appelée lorsqu'un objet est comparé à un autre à l'aide de l'opérateur `<`.

`__le__(self, other)`: Cette méthode est appelée lorsqu'un objet est comparé à un autre à l'aide de l'opérateur `<=`.

`__gt__(self, other)`: Cette méthode est appelée lorsqu'un objet est comparé à un autre à l'aide de l'opérateur `>`.

`__ge__(self, other)`: Cette méthode est appelée lorsqu'un objet est comparé à un autre à l'aide de l'opérateur `>=`.

`__add__`, `__sub__`, `__mul__`, `__div__`, etc. : Ces méthodes sont utilisées pour redéfinir les opérateurs arithmétiques (`+`, `-`, `*`, `/`, etc.) pour les objets de la classe.

